IoT Panel Troubleshooting Guide

AgenticIoT — Copilot IoT Service

Version: 1.0 | **Audience:** Copilot Studio Agent Knowledge Base

---

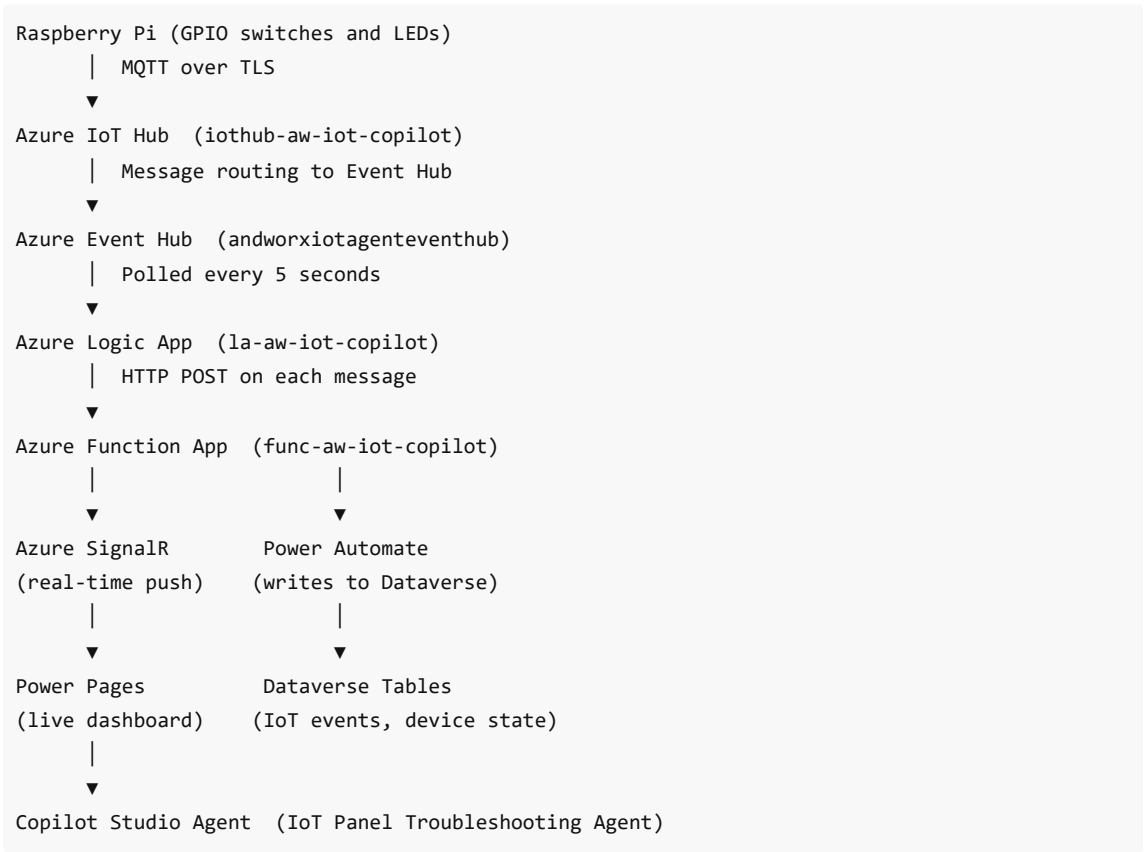
1. System Overview

The AgenticIoT system is an end-to-end IoT demonstration that connects a physical Raspberry Pi hardware panel to Microsoft Power Platform, Azure cloud services, and a Copilot Studio AI troubleshooting agent. The system demonstrates how a physical device can send real-time telemetry to a cloud dashboard, and how an AI agent can interpret that data to assist users with hardware and software diagnostics.

What the System Does

A Raspberry Pi reads 4 physical switches and controls 4 LEDs. The switch states follow configurable rules called a "logic map". When a switch state changes, telemetry is sent to Azure IoT Hub over MQTT, and the data appears in a live browser dashboard within approximately 5 to 10 seconds. If something looks wrong — a LED is on that shouldn't be, or expected LEDs are not on — the system can trigger the AI agent to help diagnose the issue.

System Architecture



Typical end-to-end latency: 5 to 10 seconds from physical switch activation to dashboard update.

Azure Resources

Resource	Name	Purpose
IoT Hub	`iothub-aw-iot-copilot`	Receives MQTT messages from the Pi
Event Hub	`andworxiotagenteventhub`	Buffers messages between IoT Hub and Logic App
Logic App	`la-aw-iot-copilot`	Polls Event Hub every 5s, POSTs to Function
Function App	`func-aw-iot-copilot`	SignalR broadcaster, telemetry endpoint
SignalR Service	`signalr-aw-iot-copilot`	WebSocket push to browser (Serverless mode)
Resource Group	`rg-aw-azcom-iot-copilot`	Contains all Azure resources

---

2. Hardware Reference

The Physical Panel

The hardware panel consists of a Raspberry Pi with four switches and four LEDs connected to its GPIO pins. The switches are numbered 1 through 4 (SW1 to SW4). The LEDs are numbered 0 through 3 (LED0 to LED3), each a different colour.

Switch types:

- **SW1** — Red-cover guarded push button (lift the red cover and turn SW1 on; lower the cover to turn SW1 off)
- **SW2** — Toggle switch (flip up to turn ON; flip down to turn OFF)
- **SW3** — Toggle switch (flip up to turn ON; flip down to turn OFF)
- **SW4** — Key switch (turn the key clockwise to turn ON; turn counter-clockwise to turn OFF)

When giving instructions to users, always use "turn SW1 on/off", "flip SW2/SW3 up/down", "turn the key on SW4 on/off". Never say "press" or "depress" for any switch.

GPIO Pin Assignments (BCM numbering)

Component	Label	BCM Pin	Physical Pin	Colour / Notes
Switch 1	SW1	GPIO 5	Pin 29	Red-cover guarded button; Pull-up; LOW (active) when turned ON
Switch 2	SW2	GPIO 6	Pin 31	Toggle switch; Pull-up; LOW when flipped ON
Switch 3	SW3	GPIO 13	Pin 33	Toggle switch; Pull-up; LOW when flipped ON
Switch 4	SW4	GPIO 19	Pin 35	Key switch; Pull-up; LOW when key turned ON
LED 0	LED0	GPIO 18	Pin 12	Blue LED
LED 1	LED1	GPIO 24	Pin 18	Orange LED

LED 2	LED2	GPIO 25	Pin 22	Green LED
LED 3	LED3	GPIO 12	Pin 32	Yellow LED

> *All LED cathodes and switch ground terminals connect to any GND pin (e.g., Pin 6, 9, 14, or 20).*

Wiring Schematic Summary

```
Switches (active-low with internal pull-up):
  GPIO 5  (BCM) ──── SW1 ──── GND
  GPIO 6  (BCM) ──── SW2 ──── GND
  GPIO 13 (BCM) ──── SW3 ──── GND
  GPIO 19 (BCM) ──── SW4 ──── GND

LEDs (active-high with 330Ω series resistor):
  GPIO 18 (BCM) ──[ 330Ω ]──[ LED0 Blue   ]──── GND
  GPIO 24 (BCM) ──[ 330Ω ]──[ LED1 Orange ]──── GND
  GPIO 25 (BCM) ──[ 330Ω ]──[ LED2 Green  ]──── GND
  GPIO 12 (BCM) ──[ 330Ω ]──[ LED3 Yellow ]──── GND
```

Component Electrical Values

Component	Value	Purpose
LED series resistor	330 Ω	Limits LED current to ~10 mA at 3.3 V
Switch pull-up	Internal PUD_UP (~50 kΩ)	Prevents floating input

> *LED current calculation: (3.3 V − 2.0 V forward voltage) / 330 Ω ≈ 4 mA. Safe for all standard 5 mm LEDs.*

---

3. Logic Map Rules

The "logic map" is the brain of the system. It defines which switch combinations should activate which LEDs. The Pi reads these rules from a JSON file called `logic_map.json`, which can be updated remotely via Azure IoT Hub Device Twin — no SSH required.

How Rules Work

1. The Pi reads all 4 switch states simultaneously every 2 seconds.
2. It checks the active switches against each rule in order.
3. The first rule that matches the active switch combination is applied: those LEDs turn ON, all others turn OFF.
4. If no rule matches, the **fallback** rule is used (LED0 only).
5. If a mismatch is detected (actual LEDs don't match expected LEDs), `needs_help` is set to `true` in the telemetry, triggering the AI agent.

> ***Important:*** *Switch numbers in the logic map are **1-based** (SW1 = GPIO 5, SW4 = GPIO 19). LED numbers are **0-based** (LED0 = GPIO 18, LED3 = GPIO 12).*

Logic Map Rules Table

Rule ID	Switches Active (1-based)	LEDs ON (0-based)	LED Colours	Description
`all_lights_on`	SW1 + SW3	LED0, LED1, LED2, LED3	Blue + Orange + Green + Yellow	Target healthy state — all 4 LEDs
`single_light`	SW2 only	LED2	Green	Single green LED
`pattern_lights`	SW1 + SW2	LED0 + LED2	Blue + Green	Two-LED pattern
`diagnostic_mode`	SW1 + SW2 + SW3	LED1 + LED3	Orange + Yellow	Diagnostic pattern
`shutdown_sequence`	SW1 + SW2 + SW4	None	All off	All LEDs extinguished
`switch_4_only`	SW4 only	LED3	Yellow	Single yellow LED
`fallback`	(no rule matched)	LED0 only	Blue	Default when no combination matches

What Each State Means

- **`all_lights_on` (SW1 + SW3):** The "healthy" demo state. All LEDs should be on. If you turn SW1 and SW3 on together and not all LEDs light up, there is a hardware or configuration problem.
- **`fallback` (no match):** Activates when the current switch combination doesn't match any rule. If you turn on switches that aren't in the rule table, the blue LED lights up by default. This is expected behaviour, not an error.
- **`diagnostic_mode` (SW1 + SW2 + SW3):** Used during demos to show a specific two-LED pattern for diagnostics discussions.
- **`shutdown_sequence` (SW1 + SW2 + SW4):** Turns all LEDs off. Use this to reset the panel to a known-off state.
- **`needs_help` flag:** Set to `true` in telemetry when the actual LED states don't match the expected LED states according to the current rule. This is the trigger for the AI agent to appear in the dashboard.

False Positives Demo Mode

The logic map includes a `false_positives` feature flag specifically for Copilot agent demos:

```
"false_positives": {
  "enabled": false,
  "probability": 0.15,
  "description": "Simulate random hardware failures for Copilot agent demo"
}
```

When `false_positives.enabled` is set to `true`, the Pi randomly fails to light expected LEDs with 15% probability per LED per reading. This creates realistic-looking hardware failure scenarios that the AI agent can diagnose. Enable via Azure IoT Hub Device Twin — no restart needed.

---

4. Common Troubleshooting Scenarios

Scenario A: "The dashboard is not showing any data"

Symptoms: The Power Pages dashboard loads but shows no switch or LED state. The "Live" or "Connected" indicator is absent or shows disconnected.

Likely causes (in order of likelihood):

1. **SignalR connection failed** — The browser failed to negotiate a WebSocket connection with the Azure Function App.
   - Check the browser console (F12) for connection errors on the `/api/negotiate` request.
   - Verify the Azure Function App is running: `https://func-aw-iot-copilot.azurewebsites.net/api/health` should return `{"status": "ok"}`.
2. **Logic App is stopped or failing** — The Logic App polls Event Hub every 5 seconds. If it's stopped, no messages reach the Function App.
   - Check Logic App run history in Azure Portal → Logic App `la-aw-iot-copilot` → Overview → Run history.
   - Runs should appear every 5 seconds. If the run history shows failures, check the Event Hub connection.
3. **Pi is not sending messages** — The Pi service may be stopped or the IoT Hub connection may be broken.
   - SSH into the Pi: `ssh pi@iotpanel.local`
   - Check service: `sudo systemctl status iot-monitor`
   - Check logs: `sudo journalctl -u iot-monitor -f`
4. **No switch activity** — The Pi only sends telemetry on switch state changes. If no switches have been activated since the service started, the dashboard shows no data.
   - Turn any switch on or off to generate a telemetry event.

Resolution steps:

1. Activate a switch on the Pi (turn SW1 on, for example).
2. Wait 5 to 10 seconds for the data to propagate.
3. If still nothing, check the health endpoint, then the Logic App run history, then the Pi logs.

Scenario B: "A specific LED is not turning on when it should"

Symptoms: You activate switches that match a rule, but one or more expected LEDs don't light up.

Questions to ask:

- Which switches are currently active (turned on)?
- Which LEDs are currently on?
- Which rule should be firing according to the logic map?

Likely causes:

1. **Wrong rule matched** — Double-check the switch combination against the rules table above. Remember: rules are matched in order, and the first match wins. SW1 + SW2 + SW3 matches `diagnostic_mode`, not `pattern_lights` (which requires only SW1 + SW2).
2. **LED hardware failure** — If the rule is correct but one LED is still off:
   - Check the LED wiring — confirm the anode (long leg) connects through a 330 Ω resistor to the GPIO pin.

    - Check the resistor is not open-circuit.
    - Test with a multimeter: GPIO pin should go to ~3.3 V when LED should be on.
3. **False positives mode enabled** — If `false_positives.enabled` is `true` in the Device Twin, the Pi intentionally simulates LED failures. Check the Twin config and set `enabled` to `false` if this is not a demo scenario.
4. **Wrong GPIO wiring** — The Pi code uses BCM pin numbering. Verify the physical wire connects to the correct BCM pin, not the physical board pin number. For example, LED0 connects to BCM GPIO 18 (physical Pin 12), not physical Pin 18.

Resolution for hardware:

- Run the hardware smoke test: `curl http://localhost:8080/api/status` while pressing switches and verify the reported `actual_leds` matches expectations.
- If `actual_leds` in the API is correct but the physical LED doesn't light, the problem is in the physical wiring.
- If `actual_leds` is wrong, the problem is in the software (check `logic_map.json`) or the GPIO read is incorrect.

Scenario C: "The Pi is offline or not connecting to IoT Hub"

Symptoms: No telemetry is arriving at the Azure IoT Hub. The dashboard may show a last-known state rather than live data.

Diagnose from the Pi:

```
# Check service status
sudo systemctl status iot-monitor

# Watch live logs
sudo journalctl -u iot-monitor -f

# Check local REST API is responding
curl http://localhost:8080/api/health
```

Log messages and their meanings:

Log Message	Meaning
`Connected to Azure IoT Hub`	Normal — IoT Hub connection established
`IoT Hub connect failed`	Connection string is wrong or IoT Hub is unreachable
`.env not found`	Credentials file is missing at `/opt/iot-monitor/.env`
`Permission denied: /dev/gpiomem`	Service user is not in the `gpio` group
`Config synced from Device Twin`	Device Twin was read successfully on startup
`Successfully sent message to Hub`	Telemetry was sent (normal)

Common fixes:

Problem	Fix
Service not started	`sudo systemctl start iot-monitor`
Bad connection string	Update `/opt/iot-monitor/.env` with correct string
Service user GPIO permission	`sudo usermod -aG gpio <service-user>` then reboot
`RPi.GPIO` not installed	`pip install RPi.GPIO` in the virtual environment
Wi-Fi disconnected	`iwconfig` or `nmcli d` to check network; reconnect to Wi-Fi
IoT Hub not reachable	Check Azure portal — IoT Hub `iothub-aw-iot-copilot` should be running

Get the IoT Hub connection string:

```
az iot hub device-identity connection-string show \
  --hub-name iothub-aw-iot-copilot \
  --device-id raspberry-pi-iotpanel \
  --query connectionString \
  --output tsv
```

Scenario D: "The dashboard updates when I activate switches but not from the Pi"

Symptoms: The test button or local simulation works fine, but physical Pi switch activations don't produce dashboard updates.

Diagnostic steps:

1. **Verify the Pi service is running and connected:**

   ```
   sudo journalctl -u iot-monitor -f
   ```

   Turn a switch on and look for: `Successfully sent message to Hub`

2. **Verify the message reaches IoT Hub:** On your dev machine:

   ```
   az iot hub monitor-events --hub-name iothub-aw-iot-copilot --device-id raspberry-pi-iotpanel
   ```

   Turn a switch on and check if a message appears here.

3. **Verify the Logic App is picking up the message:**

   - Azure Portal → Logic App `la-aw-iot-copilot` → Overview → Run history
   - A successful run should appear within 5 seconds of the switch state change.
   - If the run shows the message content, the Logic App is working.

4. **Verify the Function App received the POST:**

   - Azure Portal → Function App `func-aw-iot-copilot` → Functions → telemetry → Monitor
   - A successful invocation should appear after the Logic App run.

5. **Verify SignalR broadcast:**

- Call `/api/test-signalr` on the Function App to send a test message manually.
- If the dashboard updates from this test but not from real Pi data, the issue is in the Logic App → Function pipeline.

Scenario E: "The service keeps crashing or restarting"

Symptoms: `systemctl status iot-monitor` shows `Active: failed` or `Active: activating (restart)`.

Check crash reason:

```
journalctl -u iot-monitor -n 50 --no-pager
```

Common crash causes:

Crash symptom	Likely cause	Fix
`ModuleNotFoundError: RPi.GPIO`	GPIO library not installed	`pip install RPi.GPIO`
`RuntimeError: Failed to add edge detection`	GPIO already in use by another process	Reboot to release GPIO locks
`ConnectionRefusedError`	IoT Hub unreachable at startup	Check network and re-run service
`json.JSONDecodeError`	`logic_map.json` is corrupted	Restore from Device Twin or repo
`FileNotFoundError: /opt/iot-monitor/.env`	`.env` file missing	Create from `.env.template`

Force config reload without restart:

```
sudo touch /opt/iot-monitor/raspberry-pi/logic_map.json
```

The main loop checks file modification time every 20 seconds and reloads automatically.

Hard reset the service:

```
sudo systemctl stop iot-monitor
sudo systemctl start iot-monitor
```

Scenario F: "Switch state changes show on dashboard but rule name is wrong"

Symptoms: The telemetry arrives and updates the dashboard, but `active_rule` shows `fallback` when you expect a specific rule, or shows the wrong rule.

Cause: The switch combination active does not match any rule in the current `logic_map.json`, or the rules were overwritten by an incomplete Device Twin update.

Check current rules:

```
# On the Pi
cat /opt/iot-monitor/raspberry-pi/logic_map.json
```

Verify the Device Twin has the correct rules: In Azure Portal → IoT Hub → Devices → `raspberry-pi-iotpanel` → Device Twin → look at `desired.logic_map.rules`.

Common mistake: The `all_lights_on` rule requires **SW1 and SW3** (switches 1 and 3), not SW1 and SW2. SW1 + SW2 fires `pattern_lights` (blue + green only).

---

5. Raspberry Pi Service Management

Service Commands

```
# IoT Monitor (main GPIO + telemetry service)
sudo systemctl status iot-monitor        # Check current status
sudo systemctl start  iot-monitor        # Start the service
sudo systemctl stop   iot-monitor        # Stop the service
sudo systemctl restart iot-monitor       # Restart the service
sudo journalctl -u iot-monitor -f        # Follow live logs (Ctrl+C to exit)

# Auto-Deploy service (pulls latest code from GitHub on every boot)
sudo systemctl status iot-auto-deploy
sudo journalctl -u iot-auto-deploy -f
cat /var/log/iot-monitor/autodeploy.log
```

Expected Startup Log Sequence

A healthy startup looks like this (in order):

```
Loaded environment from /opt/iot-monitor/.env
Connected to Azure IoT Hub
Config synced from Device Twin (version 1)
Reloading subsystems with Device Twin config…
Starting monitoring loop…
```

If `Connected to Azure IoT Hub` never appears, the IoT Hub connection string is wrong or the network is down.

Local REST API

The Pi runs a local Flask REST API (default port 8080) for health checks and state inspection:

```
# Health check
curl http://localhost:8080/api/health
# Returns: {"status": "ok", "simulation_mode": false}

# Current panel state
curl http://localhost:8080/api/status
# Returns: {"switches": [1, 0, 1, 0], "leds": [1, 1, 1, 1], "active_rule": "all_lights_on",
...}
```

The `simulation_mode` field in the health check indicates whether the service is running in demo mode (false positives enabled).

Updating Code on the Pi

```
cd /opt/iot-monitor
git pull origin main
sudo systemctl restart iot-monitor
```

The auto-deploy service handles this automatically on every boot. For an immediate update, run manually as shown above.

---

6. Azure Cloud Pipeline

Message Flow Detail

1. A switch state change is detected by the Raspberry Pi GPIO service.
2. The Pi publishes a telemetry message to Azure IoT Hub `iothub-aw-iot-copilot` via MQTT over TLS.
3. An IoT Hub message route sends the message to Event Hub `andworxiotagenteventhub`.
4. Logic App `la-aw-iot-copilot` polls the Event Hub consumer group `$Default` every 5 seconds.
5. When a message is found, the Logic App sends an HTTP POST to the Azure Function App `/api/telemetry`.
6. The Function App broadcasts the telemetry via SignalR to all connected browser clients.
7. The Power Pages dashboard receives the WebSocket message and updates the UI.
8. In parallel, Power Automate writes the event to Dataverse for history and agent queries.

Azure Function Endpoints

Method	Path	Auth	Purpose
GET	`/api/negotiate`	Function key	SignalR connection info for browser clients
POST	`/api/telemetry`	Function key	Receives telemetry from Logic App
GET	`/api/test-signalr`	Anonymous	Push a test message manually
GET	`/api/health`	Anonymous	Health check

Health check URL: `https://func-aw-iot-copilot.azurewebsites.net/api/health`

Test SignalR URL: `https://func-aw-iot-copilot.azurewebsites.net/api/test-signalr`

SignalR Message Contracts

`SendTelemetryUpdate` — Sent on every telemetry message. Browser subscribes to this for real-time panel state.

```
{
  "deviceId": "raspberry-pi-iotpanel",
  "timestamp": "2026-05-18T20:00:00Z",
  "data": {
    "switches": [1, 0, 1, 0],
    "actual_leds": [1, 1, 1, 1],
```

```
    "active_rule": "all_lights_on",
    "mismatch": false,
    "needs_help": false
  },
  "source": "iot-hub"
}
```

`TriggerAgentHelp` — Sent when `needs_help` is `true` (mismatch detected). Triggers the Copilot Studio agent in the dashboard.

```
{
  "deviceId": "raspberry-pi-iotpanel",
  "timestamp": "2026-05-18T20:00:00Z",
  "active_rule": "fallback",
  "switches": [1, 0, 0, 0],
  "expected_leds": [0],
  "actual_leds": [0],
  "mismatch": false
}
```

When `needs_help` is True

The `needs_help` flag is set to `true` in the telemetry payload under two conditions:

1. The `actual_leds` reported by the Pi do not match the `expected_leds` for the current rule (a mismatch occurred).
2. The `false_positives` feature is enabled and triggered a simulated failure.

When the dashboard receives `needs_help: true`, the "Help Now" button appears, allowing the user to invoke this agent.

---

7. Configuration Reference

Device Twin Schema

The Device Twin desired properties configure the Pi's behaviour without requiring SSH or code changes. The `logic_map` key in `desired` properties controls everything:

```
{
  "desired": {
    "logic_map": {
      "version": 1,
      "description": "AgenticIoT demo panel config",
      "rules": [ ... ],
      "fallback": { "leds": [0] },
      "false_positives": {
        "enabled": false,
        "probability": 0.15
      },
      "web_ui": {
        "enabled": true,
```

```
        "port": 8080,
        "host": "0.0.0.0"
      },
      "iot_hub": {
        "enabled": true,
        "connection_string_env": "IOT_HUB_CONNECTION_STRING",
        "device_id": "raspberry-pi-iotpanel"
      }
    }
  }
}
```

Version bump: Always increment `version` when pushing a Device Twin update. The Pi logs the version number when it syncs, making it easy to confirm the update was received.

Environment File (`.env`)

Located at `/opt/iot-monitor/.env` on the Pi. Contains a single variable:

```
IOT_HUB_CONNECTION_STRING=HostName=iothub-aw-iot-copilot.azure-devices.net;DeviceId=raspberry-pi-iotpanel;SharedAccessKey=<base64key>
```

This file is gitignored. It is never committed to the repository.

logic_map.json (Local Copy)

The Pi keeps a local copy of the logic map at `/opt/iot-monitor/raspberry-pi/logic_map.json`. This file is:

- Loaded on service startup
- Updated from the Device Twin on every connection
- Hot-reloaded every 20 seconds if the file modification time changes

---

8. Zero-Touch Provisioning

For new Raspberry Pi hardware, the project supports fully automated provisioning using Azure Device Provisioning Service (DPS). This allows a new Pi to be set up without any manual SSH or credential entry on the device itself.

Provisioning Flow

```
Dev machine: Flash SD card → Run New-PiBootConfig.ps1 → Eject SD
Pi: Boot → firstrun.sh → Downloads bootstrap → Writes .env → Installs service → Reboots
Azure: Device Twin pushed → Pi picks up config on first connect
```

Required Azure Resources

- DPS instance: `dps-aw-iot-copilot` (resource group `rg-aw-azcom-iot-copilot`)
- IoT Hub: `iothub-aw-iot-copilot` linked to DPS
- DPS group enrollment: `iotpanel-fleet`

Provisioning Steps

Step 1: Flash Raspberry Pi OS Lite (64-bit) using Raspberry Pi Imager. Set hostname to `iotpanel` , enable SSH with password, configure Wi-Fi.

Step 2: Get DPS credentials:

```
$scope = az iot dps show --name dps-aw-iot-copilot --resource-group rg-aw-azcom-iot-copilot
--query properties.idScope -o tsv
$key = az iot dps enrollment-group show --dps-name dps-aw-iot-copilot --resource-group rg-
aw-azcom-iot-copilot --enrollment-id iotpanel-fleet --show-keys --query
attestation.symmetricKey.primaryKey -o tsv
```

Step 3: Write credentials to the SD card:

```
.\scripts\New-PiBootConfig.ps1 -DriveLetter E -IdScope $scope -GroupKey $key
```

Step 4: Insert SD card into Pi and power on. Wait approximately 5 minutes.

Step 5: Verify:

```
ssh pi@iotpanel.local
cat /var/log/iot-firstrun.log
sudo systemctl status iot-monitor
```

---

9. Diagnostic Commands Quick Reference

On the Raspberry Pi

```
# Service status
sudo systemctl status iot-monitor

# Live log tail
sudo journalctl -u iot-monitor -f

# Last 50 log lines
sudo journalctl -u iot-monitor -n 50 --no-pager

# Local health check
curl http://localhost:8080/api/health

# Local panel state
curl http://localhost:8080/api/status

# Force config reload
sudo touch /opt/iot-monitor/raspberry-pi/logic_map.json

# Manual run for debugging (stops service first)
sudo systemctl stop iot-monitor
cd /opt/iot-monitor/raspberry-pi
python3 main.py

# Restart service
sudo systemctl restart iot-monitor
```

```
# Update code
cd /opt/iot-monitor && git pull origin main && sudo systemctl restart iot-monitor
```

Azure CLI (from dev machine)

```
# Monitor live IoT Hub events from the Pi
az iot hub monitor-events --hub-name iothub-aw-iot-copilot --device-id raspberry-pi-iotpanel

# Check device connection state
az iot hub device-identity show --hub-name iothub-aw-iot-copilot --device-id raspberry-pi-iotpanel --query connectionState

# Get device connection string
az iot hub device-identity connection-string show --hub-name iothub-aw-iot-copilot --device-id raspberry-pi-iotpanel --query connectionString --output tsv

# Update Device Twin to enable false positives (demo mode)
az iot hub device-twin update --hub-name iothub-aw-iot-copilot --device-id raspberry-pi-iotpanel --desired '{"logic_map": {"version": 2, "false_positives": {"enabled": true, "probability": 0.15}}}'

# Disable false positives
az iot hub device-twin update --hub-name iothub-aw-iot-copilot --device-id raspberry-pi-iotpanel --desired '{"logic_map": {"version": 3, "false_positives": {"enabled": false}}}'
```

Azure Function App (HTTP)

```
# Health check
curl https://func-aw-iot-copilot.azurewebsites.net/api/health

# Push a manual test SignalR message to the dashboard
curl https://func-aw-iot-copilot.azurewebsites.net/api/test-signalr
```

---

10. Hardware Smoke Test Procedure

Use this procedure to verify hardware is correctly wired after assembly or after any physical changes.

Step 1: Confirm service is running.

```
systemctl status iot-monitor
```

Step 2: Activate each switch and observe:

Action	Expected LED(s) on
Turn SW2 ON only	LED2 (Green)

Turn SW4 ON only	LED3 (Yellow)
Turn SW1 ON only	LED0 (Blue) — fallback rule
Turn SW1 + SW3 ON	LED0 + LED1 + LED2 + LED3 (all 4)
Turn SW1 + SW2 ON	LED0 + LED2 (Blue + Green)
Turn SW1 + SW2 + SW3 ON	LED1 + LED3 (Orange + Yellow)
Turn SW1 + SW2 + SW4 ON	No LEDs on

Step 3: Check live logs during switch activation:

```
sudo journalctl -u iot-monitor -f
```

You should see `Switch state change → telemetry sent (rule: <rule_id>)` for each change.

Step 4: Check the local API:

```
curl http://localhost:8080/api/status
```

The `switches` and `leds` arrays should reflect the current physical state.

Step 5: If a specific LED doesn't light up but the software shows it should:

- Check the 330 Ω resistor is in series (not short-circuited).
- Verify the LED is connected anode → resistor → GPIO pin, cathode → GND.
- Measure the GPIO pin voltage with a multimeter — it should be ~3.3 V when on, ~0 V when off.
- Try a replacement LED (LEDs can fail open-circuit).

---

11. Escalation Guide

When to Escalate

Escalate to a human engineer when:

- The hardware smoke test fails and LED replacement does not fix the problem (possible damaged GPIO pin).
- The Pi service repeatedly crashes despite correct `.env` and proper dependencies.
- The Azure IoT Hub shows the device as disconnected and all credentials are verified correct.
- The Logic App shows consistent failures in run history with 5xx errors to the Function App.
- The Function App is returning errors and a restart has not resolved it.

How to Escalate

1. Capture the relevant logs: `journalctl -u iot-monitor -n 100 --no-pager > /tmp/iot-monitor-logs.txt`
2. Note the current Device Twin version.
3. Screenshot the Logic App run history showing the failure.
4. Note the exact switch combination pressed and expected vs. actual LED states.
5. File an issue at: `https://github.com/Andworx/copilot-iot-service/issues`

This document was generated from the AgenticIoT project documentation to serve as knowledge for the IoT Panel Troubleshooting Copilot Studio Agent.